

LECTURE-23

Thursday
7/6/20

□ Optimal Page Replace Algorithm: Gives best no. of Hit ratio but cannot be used practically.

- We'll replace the page here jo future main sabse baad main use hoga. Since future main dekhne hai $\therefore$ very difficult & uncertain.
- Used for comparison with other algs.

means 7 isn't in queue.

in future as comparison to 0 and 1

7 replaced with 2 as 7 sabsey end pr use horha ↑

	F	F	F	F	H	F	H	F	H	H	F	H	H	H	F	H	H				
	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1	7
F1	7	7	7	2	2	2	2	2	2	2	2	2	2	2	2	2	2	7	7	7	7
F2		0	0	0	0	0	0	4	4	4	0	0	0	0	0	0	0	0	0	0	0
F3			1	1	1	3	3	3	3	3	3	3	3	1	1	1	1	1	1	1	1

1 replaced with 3 as '0' came right after 3, 2 came after

Hits = 11, Faults = 9. Three inputs of 3 and 1 came after 8 inputs

☐ Least Recently Used (LRU): Replaces the page that has not been used for longest time

we'll see the input which is in past of '3' i.e.: 7 0 1 2 0

8 now we'll replace with oldest value either 2, 0, 1 which is 1

	F	F	F	F	H	F	H	F	F	F	F	F	F	F	F	F	F	F	F	F
	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
F1	7	7	7	2	2	2	2	4	4	4										
F2		0	0	0	0	0	0	0	0	3	...	...	...	...	...	...	...	...	...	...
F3			1	1	1	3	3	3	2	2										

☐ Second Chance Algorithm / Clock: Give a page second chance before replacement.

• For keeping track of chances, we'll attach a bit with frames.

• Bit = 1; change bit to '0' (chance given but not replaced.)

Bit = 0; page replaced as chances finished.

Bit = 1; this will be done when any page gets 'Hit' when bit = 0.

• Keep a pointer that will move to next frame after each replacement. Ham current pointer wala frame ki bit check kreingay.

This is the pointer; it tells kay kis frame say replace krna hai. Agar pointer walaay frame kay page ki bit '0' hogi tab replace hoga werna pointer moves

This is the chance Bit. Ager 1 hoga to '0' hojayege and won't be replaced. '0' hogi to 1 hojaye ga if it comes in input

	F	F	F	F	H	F	H	F	F	F	H	H	F
	7	0	1	2	0	3	0	4	2	3	0	3	2
F1	7 ₀	7 ₀	7 ₀	2 ₀	2 ₀	2 ₀	2 ₀	4 ₀	4 ₀	3 ₀	3 ₀	3 ₁	3 ₁
F2		0 ₀	0 ₀	0 ₀	0 ₁	0 ₀	0 ₁	0 ₁	0 ₀	0 ₀	0 ₁	0 ₁	1 ₀
F3			1 ₀	1 ₀	1 ₀	3 ₀	3 ₀	3 ₀	2 ₀	2 ₀	2 ₀	2 ₀	2 ₁

Method:

- Since the pointer was in F1 and the bit of '7' was '0' $\therefore$ 2 replaced 7 and its current bit (i.e. importance is zero) ^{pointer moved to next frame} Then when zero came as an input; the importance of zero which was already in F2 increased $\therefore$ we increased its bit from zero to 1. Pointer stayed at F2 as on hit pointer does not change frames. Then 3 came in input which gave '0' a chance to stay as its bit was 1 and bit ^{of zero} changed to zero and in order to fit new input '3'; pointer moved to F3 & replace 1 as its bit was zero. In second last column; when all three bits are 1; one by one all gets zero and no. is replaced in the same frame.

Copy-on-write (COW):

- When we use to make childs from fork() but then we call exec(); then entire child was over written to our samai mehnat from parent & time gets zaya.
- Therefore we'll share the PCB at first instead of making a copy like before (which use to get over written by exec).
- Now whenever the child variable memory is over written so in order to make separate copies of child & parent; we'll just make a copy of specific page jahan no over written variable para hai. This copying of page is called 'COW'.

Vfork(): Used in very specific situations

- Child process is created by using the address space of parent instead of a new address space. Parent will remain suspended till then.
- This is cheaper than Fork().